

Constructors, Destructors

Constructor: is a special setup function that runs automatically the exact moment an object is created. Its sole job is to establish the object's initial state so it is immediately ready for use.

The constructor has:

- The same name as the class
- No return type, not even void

1. Default Constructor: Initializes an Object with predefined values. It runs automatically if no arguments are provided during instantiation.

```
Car() {  
    this->brand = "Generic Car";  
    this->speed = 0;  
}  
// OR  
Car() : name("Generic Car"), speed(0) {}
```

2. Parameterized Constructor: Initializes a Car with specific data provided by the user. It allows you to assign unique values to an object's attributes at the exact moment of its birth.

```
Car(string brand, int speed) {  
    this->brand = brand;  
    this->speed = speed;  
}  
// Usage: Car myCar("Tesla", 120);
```

Under the Hood: Notice the “: name("Generic Car"), speed(0)” syntax in the code above. This is an **Initializer List**. In C++, if you assign variables inside the curly braces {}, the compiler actually builds the variables with default garbage values first, and then overwrites them. Initializer lists construct the data directly with the correct value on the first pass, saving CPU cycles.

Notes:

The Implicit Default: If you write **no** constructors, C++ provides a "hidden" default one. It allocates memory but doesn't initialize variables, leaving them with **garbage values** (random leftover data in memory).

The Deletion Rule: The moment you define **any** constructor (like a Parameterized one), C++ assumes you want full control. It **permanently deletes** the hidden default constructor.

The Fix: If you still want to create an object without passing values (e.g., Car myCar;), you must now manually write the default constructor back into your code.

Destructors

A **Destructor** is a special cleanup function that runs automatically the exact moment an object is destroyed (goes out of scope or is explicitly deleted). Its only job is to free up any external resources the object was holding, like heap memory, file handles, or network sockets.

The destructor has:

- It has the same name as the class but is preceded by a tilde (~).
- It takes no arguments, no return type, and cannot be overloaded.

```
~Car() {  
    cout << "Car object destroyed and memory freed." << endl;  
}
```

Under the Hood: The C++ compiler guarantees that destructors are called in the exact **reverse order** of constructors (Stack unwinding).

```
Car car1; // 1st constructed  
Car car2; // 2nd constructed  
Car car3; // 3rd constructed  
// Destructors run: car3 is destroyed first, then car2, then car1.
```

Note: If a class has any virtual functions, its destructor **must** be declared virtual. Why? Because in polymorphism, you often point to a Child object using a Parent pointer (**Parent* p = new Child();**). If you call delete p; and the parent's destructor is not virtual, the compiler only destroys the Parent half of the object. The Child half is left stranded in RAM forever, causing a massive memory leak.